

📖 OOAD Ch11: Refactoring、Polymorphism 與 State Pattern

主題重點： 從 [Blockbuster](#) 錄影帶出租系統案例理解「爛設計」為何難以維護，透過 [Refactoring](#) 逐步集中變動邏輯，最後用 [State Design Pattern](#) 解決電影價格分類會隨時間轉換的問題。

📺 課程簡介：百視達錄影帶出租系統

這堂課從一個「電影出租系統」的小練習開始。系統要處理三種電影分類：

- **New release**（新片）：定價 \$3
- **Regular**（舊片 / 常規片）：定價 \$2
- **Children**（兒童片）：定價 \$2

系統的基本邏輯包含：

- 一部電影可以有多个副本。
- 客戶可以租借電影。
- 客戶可以要求列印出租帳單（`statement()`）。

在初始的 [UML](#) 概念中，至少會出現三個類別：

- **Movie**
 - 屬性： `priceCode`
- **Rental**
 - 屬性： `daysRented`
- **Customer**
 - 方法： `statement()`

類別間的關聯為：

- 一個 **Customer** 對應多個 **Rental** 紀錄。
- 一個 **Rental** 對應一部 **Movie**。

📄 初始程式碼分析： `movie-p05.java`

老師提供了一份名為 `movie-p05.java` 的初始程式碼。雖然它已經有類別宣告，但老師評價為：

not well designed, poorly written
設計不佳、寫得很爛。

程式碼結構

初始版本的核心問題集中在 `Customer.statement()`：

- **Customer** 內有一個負責產生帳單的 `statement()` 方法。
- `statement()` 會用迴圈抓取所有 **Rental** 物件。
- 再透過 **Rental** 取得：
 - `Movie.priceCode`
 - `Rental.daysRented`
- 然後在 `Customer.statement()` 裡面直接計算：
 - 每筆租借費用
 - 總費用
 - 常客積分（frequent renter points）
 - 帳單輸出字串

其中最明顯的 bad smell 是：

```
switch (priceCode) {  
    // different movie types charging logic  
}
```

JAVA

也就是說，`Customer.statement()` 同時負責：

1. 走訪租借資料
2. 判斷片型
3. 計算金額
4. 計算積分
5. 組出輸出格式

這違反了良好設計中「責任分離」與「高內聚、低耦合」的精神。

⚠️ 初始設計面臨的維護問題

1. 幾乎無法支援新輸出格式

假設客戶要求新增：

```
htmlStatement()
```

JAVA

也就是除了純文字帳單以外，還要輸出 HTML 格式帳單。

程式設計師會發現：

- 現有的 `statement()` 幾乎無法重複使用（reuse）。
- 因為計費邏輯、積點邏輯與字串輸出格式混在一起。
- 最快的做法只能是 copy-paste `statement()`，再修改輸出字串。

這會導致重複邏輯（duplicated logic）。

2. 一致性維護困難

如果未來：

- 常客積點規則改變
- 計費邏輯改變
- 新增促銷方案

開發者就必須同時修改：

- `statement()`
- `htmlStatement()`

如果只改其中一個，就會造成兩種帳單格式計算結果不一致。

3. 軟體熵（Software Entropy）

老師用這個案例說明 [Software Entropy](#)：

- 糟糕的程式碼一開始也許可以跑。
- 但隨著需求變更、人員更迭與時間推移，修改會越來越困難。
- 每次新需求都需要改越來越多地方。
- 改完之後又要花更多時間 debug 與測試。

當：

修改 + 除錯 + 測試的成本 > 重做（rework）的成本

系統就會進入必須打掉重練的狀態。

🔍 評估系統設計：重點不是唯一正解，而是未來變動

老師強調：

沒有任何一個設計是絕對正確的答案。

判斷設計好壞的標準，不是看它現在能不能跑，而是看它是否能對付未來可能發生的維護需求。

本案例預期會變動的地方包括：

1. 片子的種類會新增與刪減
 - 例如：3D 影片、成人片等。
 - 老師認為這幾乎一定會發生。
2. 常客積點與促銷邏輯會更改
 - 這屬於 [Business Logic](#)。
3. 帳單格式會不斷修改
 - 文字格式、HTML 格式、甚至其他格式。
4. 片子的價格會改變
5. 一部片子會從新片變成舊片
 - 這是後面討論錯誤繼承設計與 State Pattern 的關鍵。

設計的核心目標是：

把未來容易變動的程式碼集中化處理（centralize）。

🔧 Refactoring 過程詳解

[Refactoring](#) 是在不改變外部行為的前提下，一步一步改善既有程式碼結構的技術。

老師強調：

- 重構不是一次全部重寫。
- 每一步都應該小而可驗證。
- 每一步都需要測試案例確認行為沒有改變。
- 重構不是零成本，但能降低未來維護成本。

1 Extract Method: `movie-p11.java`

第一步是把 `Customer.statement()` 中巨大的 `switch` 計費邏輯抽出來，變成 `Customer` 的新方法：

```
amountFor()
```

JAVA

這一步的好處是：

- `statement()` 開始變短。
- 計費邏輯變成一個獨立方法。
- 後續比較容易移動與重用。

但老師指出，這一步仍然有問題：

- `amountFor()` 使用太多 `Rental` 的資料。
- 原則上，一個方法應該放在它最常使用其資料的物件上。

因此，`amountFor()` 不應該長期留在 `Customer`。

2 Move Method: `movie-p21.java`

接著將 `amountFor()` 搬移到 `Rental` 類別，並改名為：

```
getCharge()
```

JAVA

這樣責任分配會比較合理：

- `Rental` 知道自己租了幾天。
- `Rental` 可以詢問對應的 `Movie` 是什麼類型。
- 所以每筆租借的費用應該由 `Rental` 負責計算。

此時 `Customer` 變得更清爽，只需要：

- 走訪每筆 `Rental`
- 呼叫 `rental.getCharge()`
- 組出帳單內容

同時也可以移除不必要的暫存變數（remove temp variables），避免方法變得過長、難讀。

3 處理常客積點: `movie-p25.java`

下一步是把常客積點（frequent renter points）的邏輯也抽出來，搬到 `Rental` 類別：

```
getFrequentRenterPoints()
```

JAVA

這樣 `Customer.statement()` 不再需要知道積點怎麼算，只要：

```
frequentRenterPoints += rental.getFrequentRenterPoints();
```

JAVA

這一步同樣是在集中 business logic，降低未來重複修改的風險。

4 加入新格式支援: `movie-p33.java`

重構到這個階段後，新增 `htmlStatement()` 就變得可行。

原因是：

- 計費邏輯已經集中在 `getCharge()`。
- 積點邏輯已經集中在 `getFrequentRenterPoints()`。
- `statement()` 與 `htmlStatement()` 只需要負責不同的輸出格式。

即使 `htmlStatement()` 在文字組裝上與 `statement()` 有部分相似，也不會造成嚴重維護問題，因為真正關鍵的 business logic 沒有重複。

也就是說：

重複輸出格式的程式碼可以接受，但不能重複核心計算邏輯。

5 消除 Bad Smell: `movie-p37.java`

雖然 `Rental.getCharge()` 比原本好，但仍然有一個問題：

- `Rental.getCharge()` 依賴 `Movie.priceCode` 來做 switch 判斷。

這代表 `Rental` 仍然知道太多 `Movie` 內部的定價分類。

如果未來新增「成人片」：

- 要改 `Movie` 的 price code。
- 也要改 `Rental.getCharge()` 裡面的 switch。
- 常客積點邏輯可能也要跟著改。

解法是把：

- `getCharge()`
- `getFrequentRenterPoints()`

搬到 `Movie` 類別，讓 `Rental` 只做委派（delegation）：

```
class Rental {
    double getCharge() {
        return movie.getCharge(daysRented);
    }
}
```

JAVA

這樣 `Rental` 不再需要理解不同片型的計費方式。

🔗 多型與繼承的陷阱

即使將方法移到 `Movie` 類別，程式碼仍然不完美，因為 `switch` 還存在。

在物件導向中，這類根據型別分支的 `switch`，常常可以用：

- [Inheritance](#)
- [Polymorphism](#)

來取代。

錯誤的直覺設計

業餘的物件導向分析師（amateur OO analysts）常會直覺設計成：

```
Movie
├─ RegularMovie
├─ ChildrensMovie
└─ NewReleaseMovie
```

然後把不同片型的計費邏輯寫在不同子類別中。

這看起來符合多型，但其實有致命缺陷。

致命缺陷：物件不能在生命週期內改變 Class

本系統有一個非常重要的需求：

一部片子會從新片變成舊片。

如果 `NewReleaseMovie` 和 `RegularMovie` 是不同 class，那代表一部電影從新片變成舊片時，理論上要從：

```
NewReleaseMovie object
```

變成：

```
RegularMovie object
```

但在物件導向中：

一個 `object` 在生命週期內不能改變自己的 `class`。

如果硬要做，就只能：

1. 建立一個新的 `RegularMovie` 物件 `p`
2. 把原本 `NewReleaseMovie` 物件 `q` 的資料複製到 `p`
3. 刪除或棄用 `q`
4. 修正所有原本指向 `q` 的 `Rental` references

這會造成大量額外程式碼，也很容易出錯。

因此，直接對 `Movie` 做繼承不是正確解法。

🧠 正確解法：State Design Pattern

為了解決「物件狀態會改變，但物件身份不能改變」的問題，老師引入 [State Design Pattern](#)。

在 `movie-p52.java` 中，設計改成：

```
Movie
└─ has a Price

Price
├─ RegularPrice
├─ ChildrensPrice
└─ NewReleasePrice
```

實作方式

不直接讓 `Movie` 繼承出不同子類別，而是建立一個額外的抽象類別：

```
abstract class Price
```

JAVA

再由 `Price` 衍生出：

- `RegularPrice`
- `ChildrensPrice`
- `NewReleasePrice`

`Movie` 內部則持有一個指向 `Price` 物件的 reference：

```
class Movie {
    private Price price;
}
```

JAVA

當外部呼叫 `Movie.getCharge()` 時，`Movie` 再將責任委派給目前的 `Price` 物件：

```
double getCharge(int daysRented) {  
    return price.getCharge(daysRented);  
}
```

為什麼這樣比較好？

當一部電影從新片變成舊片時，不需要更換 **Movie** 物件本身。

只需要把：

```
Movie.price = NewReleasePrice
```

換成：

```
Movie.price = RegularPrice
```

也就是說：

- 原本的 **Movie** object 不變。
- 原本所有 **Rental** 對 **Movie** 的 reference 不變。
- 改變的是 **Movie** 內部的狀態物件 **Price**。

這正是 **State Design Pattern** 的價值：

當物件的行為取決於其狀態，且狀態會在生命週期內轉換時，將狀態抽成獨立物件。

🔗 Design Patterns 與 OOAD 總結

Design Patterns 是架構經驗的重用

Design Patterns 不是單純的 code reuse，而是：

pattern reuse, not code reuse

也就是把前人在物件導向軟體中反覆遇到的問題、繼承架構與物件互動方式整理成可重複使用的設計經驗。

OOAD 的目的

Object-Oriented Analysis and Design 的核心目的不是炫技，而是讓程式碼更容易：

- maintain (維護)
- change (改變)
- evolve (演化)

這與本章重構案例完全一致：

- 初始程式碼可以跑，但難以演化。
- 重構後的設計把變動點集中起來。
- 最後用 State Pattern 支援「電影分類會改變」這個真實需求。

📊 效能 vs 架構

老師提醒：

好的架構不等於好的效能。

在很多情況下，良好物件導向架構會透過多層委派來分離責任，例如：

```
Customer → Rental → Movie → Price
```

這讓系統更好維護，但也可能比直接用一個大 **switch** 慢一點。

有時候為了效能，甚至會把類別合併、減少委派層數。

所以設計必須取決於實際考量：

- 系統規模
- 效能需求
- 維護成本
- 未來變動機率
- 開發團隊能力

🚫 不要過度設計

老師也提醒不要 **overdesign**。

如果某塊程式碼預期未來 100 年都不會變，硬把它設計成非常複雜的 OO 架構，只是在浪費時間。

OOAD 的投入程度取決於：

- 軟體領域的演進速度
- 市場需求變化
- 系統規模
- 成本

- 團隊人員流動率
- 維護年限

例如：

- 多數研究型專案可能只需要快速驗證演算法，不一定需要完整 OOAD。
- 商業產品若會長期維護、多人接手，就更需要重視分析與設計。

✔ 本章核心觀念總結

1. 設計好壞取決於能否支援未來變動，而不是現在能不能跑。
2. 初始 `Customer.statement()` 的問題在於混合了輸出格式、計費邏輯與積點邏輯。
3. [Refactoring](#) 的目的，是在不改變外部行為的前提下，逐步改善內部結構。
4. `Extract Method` 可以先切出邏輯，但方法應該放到最常使用其資料的物件上。
5. `Move Method` 將 `getCharge()` 與 `getFrequentRenterPoints()` 移到更適合的類別，降低耦合。
6. 新增 `htmlStatement()` 的關鍵不是完全避免重複，而是避免重複 business logic。
7. 根據 `priceCode` 的 `switch` 是 bad smell，通常應該思考是否能用多型取代。
8. 直接讓 `Movie` 繼承出 `RegularMovie`、`ChildrensMovie`、`NewReleaseMovie` 是錯誤直覺，因為電影會從新片變舊片。
9. 一個 object 不能在生命週期中改變 class，因此狀態轉換不應靠更換物件類別解決。
10. [State Design Pattern](#) 透過 `Movie has-a Price`，讓電影分類可以更換而不破壞原本 references。
11. [Design Patterns](#) 是架構經驗的重用，不是程式碼片段的重用。
12. OOAD 是為了維護、改變與演化，但也要避免過度設計。

🔗 Related Notes

-  [School/OOAD_Object-Oriented Analysis and Design](#)
- [School/OOAD/CH10_Model-Based_Software_Engineering_and_UML](#)
- [School/OOAD/CH9_Problem_A_Blockbuster_Answer](#)
- [Refactoring](#)
- [State Design Pattern](#)